

Article

Selective Falsification for Budget-Aware Self-Consistency in Chain-of-Thought Math Reasoning

Firstname Lastname¹, Firstname Lastname² and Firstname Lastname^{2,*}

¹ Affiliation 1; e-mail@e-mail.com

² Affiliation 2; e-mail@e-mail.com

* Correspondence: e-mail@e-mail.com; Tel.: (optional; include country code; if there are multiple corresponding authors, add author initials) +xx-xxxx-xxx-xxxx (F.L.)

Abstract

Self-consistency (SC) improves chain-of-thought (CoT) reasoning by sampling multiple solutions and returning the most frequent final answer, but it implicitly treats sampled chains as exchangeable and uses popularity as a proxy for correctness. This can be brittle when a smaller or cost-constrained model repeatedly samples the same misconception, making the majority answer wrong. Confidence-weighted voting appears to address this but typically requires verifying every sampled chain and depends on self-reported confidence that can be poorly calibrated. We propose Selective Falsification-Weighted Self-Consistency (SFw-SC-CoT), a prompt-only aggregation rule that clusters sampled CoT outputs by unique final answer and verifies each answer hypothesis once with a deterministic “try to disprove it” checker. The checker performs three explicit sanity checks (estimation, alternative recomputation, and constraint checking) and outputs a confidence score and a checks-passed count. We combine down-weighted frequency with these falsification signals in a budget-aware scoring function. On GSM8K test problems ($N = 200$) with gpt-4o-mini and $K = 8$ sampled chains, SFw-SC-CoT increases accuracy from 0.93 to 0.94, while highlighting the need for improved compute logging to substantiate budget efficiency claims.

Keywords: keyword 1; keyword 2; keyword 3 (List three to ten pertinent keywords specific to the article; yet reasonably common within the subject discipline.)

1. Introduction

Chain-of-thought (CoT) prompting has become a standard inference-time technique for eliciting multi-step reasoning from large language models (LLMs) without fine-tuning. By prompting a model to “solve step by step” and produce intermediate natural-language reasoning, CoT can substantially improve performance on tasks that require multiple arithmetic or logical steps compared with directly prompting for the final answer, particularly at larger model scales [1]. At the same time, CoT does not guarantee correctness or faithfulness: a model can generate fluent reasoning that contains subtle arithmetic slips or incorrect assumptions, and stochastic decoding can produce different rationales and different answers for the same question [1]. This variability is especially consequential in settings where one seeks improved reliability without changing model weights, for example when only prompting is available.

Self-consistency (SC) is a widely used aggregation approach that attempts to stabilize CoT outcomes at inference time. Instead of trusting a single sampled chain, SC generates

K independent CoT samples and selects the most frequent extracted final answer via majority vote [2]. When errors are diverse and relatively uncorrelated, majority voting can outperform any single sample by amplifying the most probable correct solution path.

However, SC makes a strong implicit assumption: it treats sampled chains as exchangeable evidence and equates the frequency of an answer with its likelihood of being correct. This assumption becomes brittle when samples are correlated through shared misconceptions. In that regime, increasing K can simply increase the frequency of the same wrong answer rather than revealing an alternative correct hypothesis. A seemingly natural extension is to weight each sample's vote by a confidence score obtained from a verifier prompt. In practice, such per-sample verification can (i) depend on self-reported confidence that may be miscalibrated, and (ii) increase compute because it adds an additional verifier call for each of the K chains.

This paper targets a prompt-only gap: an aggregation rule that (a) is less sensitive to shared misconceptions than pure frequency voting by explicitly attempting falsification, and (b) can be more sample-efficient by verifying only a small number of answer hypotheses rather than verifying every sampled chain. We propose Selective Falsification-Weighted Self-Consistency (SFw-SC-CoT), a simple, inference-only algorithm that reframes SC as hypothesis testing. The method first samples K CoT solutions, clusters them by unique extracted final answer, and then runs a deterministic verifier once per unique answer hypothesis. The verifier is asked to try to disprove the hypothesis using three explicit sanity checks: a magnitude estimate, an alternative recomputation, and a constraint check based on the problem statement. It outputs two structured signals: a confidence $c(a) \in [0, 1]$ and a discrete checks-passed count $p(a) \in \{0, 1, 2, 3\}$. We then select the answer with the highest score under a budget-aware aggregation rule that down-weights frequency so that popularity alone cannot dominate.

We evaluate SFw-SC-CoT using the provided codebase on GSM8K test problems. In the executed experiment, both baseline SC and SFw-SC-CoT use the same generation budget ($K = 8$ CoT samples, temperature 0.7, max_tokens 512) and the same model (gpt-4o-mini). The baseline uses majority voting over extracted answers; the proposed method performs answer-level falsification verification and weighted aggregation. On $N = 200$ GSM8K test problems, the proposed method achieves 0.94 accuracy (188/200) compared with 0.93 (186/200) for SC.

1.1. Contributions

- **Selective falsification aggregation.** We propose SFw-SC-CoT, a prompt-only aggregation method that verifies answer hypotheses rather than individual chains, using structured falsification prompts.
- **Budget-aware scoring rule.** We introduce a scoring rule that combines down-weighted frequency with two verifier-derived signals (confidence and checks passed).
- **End-to-end evaluation.** We provide an end-to-end evaluation on GSM8K ($N = 200$) with gpt-4o-mini at fixed $K = 8$, showing a small positive accuracy improvement and reporting the corresponding runtime difference.

The current results are intentionally narrow, reflecting what was actually executed and logged. Future work should extend the evaluation across additional K values and datasets, and, crucially, log hypothesis counts and token usage so that accuracy can be assessed under explicit compute budgets rather than only under fixed sampling parameters.

2. Related Work

CoT prompting is a foundational inference-time method for improving multi-step reasoning performance in LLMs without weight updates [1]. Its primary relevance to our

setting is twofold. First, it provides the mechanism for producing explicit intermediate reasoning steps that often help models solve arithmetic word problems. Second, it exposes a reliability issue: models may produce plausible but incorrect reasoning traces, and stochastic generation can lead to multiple distinct solutions and answers for the same input [1]. These properties motivate aggregation methods that can exploit diversity across samples.

Self-consistency (SC) is a prominent aggregation strategy that samples multiple CoT chains and returns the most frequent final answer [2]. Relative to single-shot CoT, SC trades additional inference compute for improved robustness to decoding randomness. The key assumption is that the correct solution path is sampled more often than any particular incorrect path; under this assumption, majority vote increases the probability of returning the correct answer.

Our work is closest to SC but differs in both the evidential model and the compute allocation. SC effectively assumes exchangeability of sampled chains: each chain contributes one unit vote, and frequency is the sole criterion for selection. This makes SC vulnerable when samples share a systematic misconception, because correlated errors can become the majority. SFw-SC-CoT modifies the SC pipeline by treating distinct final answers as hypotheses and introducing an explicit falsification-oriented verification step at the hypothesis level. Instead of verifying each chain (which would scale verification compute linearly with K), we verify each unique answer hypothesis. This design aims to reduce the number of verifier calls from K to U , where U is the number of distinct extracted answers observed among the K samples.

SFw-SC-CoT also differs from confidence-weighted voting schemes that rely only on self-reported confidence. While both our method and confidence weighting can be implemented via prompting, the falsification prompt used in SFw-SC-CoT constrains the verifier to perform three independent sanity checks and to report a discrete checks-passed signal alongside a continuous confidence score. The intent is to make verification less dependent on a single unconstrained confidence statement and to encourage error-search behavior similar to hypothesis testing.

In the present work, our empirical comparison is limited to SC majority vote, because only that baseline was executed and logged in the available experiments. Therefore, the primary contribution of the related-work comparison is conceptual: SFw-SC-CoT can be interpreted as an SC-style sampler with an additional hypothesis-level verification stage designed to mitigate correlated error modes that frequency alone cannot resolve.

3. Background

This work focuses on multi-step math word problems where the desired output is a single numeric answer. The overall pipeline consists of chain-of-thought generation, extraction and normalization of numeric answers, aggregation across multiple samples, and (for the proposed method) verification signals derived from a falsification-style prompt.

3.1. Problem setting and notation

Let q denote a natural-language problem. A stochastic CoT generator produces K samples. Each sample i yields a rationale r_i (the generated reasoning text) and an extracted final numeric answer a_i . Samples are grouped by identical extracted answers. For any candidate answer string a , define $G(a)$ as the set of rationales that end in a , that is, $G(a) = \{r_i : a_i = a\}$. Let U be the number of unique answers with non-empty groups.

3.2. Baseline self-consistency

Standard SC aggregates by majority vote over extracted final answers, returning $\arg \max_a |G(a)|$ [2]. In effect, it estimates that an answer is correct if it is sampled most frequently among the K chains.

3.3. Answer extraction and normalization

The implementation extracts a final numeric answer from a rationale using a prioritized list of regular expressions. These patterns are designed to handle common answer formats observed in practice, including GSM8K-style boxed numbers like “#### 42”, explicit Answer phrases, markdown bold numbers, and other answer formats, with a final permissive fallback that selects the last number appearing in the text.

Ground-truth labels are extracted from GSM8K solutions primarily using the #### delimiter. For evaluation, predicted and ground-truth answers are normalized by removing commas and spaces, stripping certain symbols like the dollar sign, and converting to a canonical numeric string. This allows integer and decimal representations to compare as equal, for instance 5 and 5.0.

3.4. Answer-hypothesis verification signals

SFw-SC-CoT introduces verification at the level of answer hypotheses. For each candidate answer a , a verifier prompt is constructed from the problem q , the proposed answer a , and up to L representative rationales from $G(a)$. Representatives are selected by sorting rationales by length and choosing the shortest ones, reducing prompt length and limiting reliance on a single verbose chain. The verifier is instructed to attempt falsification via three sanity checks: (1) estimation or order-of-magnitude reasoning, (2) a brief alternative recomputation, and (3) a constraint check for consistency with the problem statement. The verifier output is parsed to obtain two scalars: confidence $c(a) \in [0, 1]$ and checks_passed $p(a) \in \{0, 1, 2, 3\}$. These signals serve as inputs to the proposed aggregation rule.

4. Method

SFw-SC-CoT modifies standard self-consistency by adding a falsification-oriented verification stage at the answer-hypothesis level. The method is inference-only and prompt-only: it uses no fine-tuning, no external tools, and no additional training data.

Given a problem q , the method proceeds as follows.

First, it samples K chain-of-thought completions with stochastic decoding, producing rationales $r_1, \dots, r_K$. In the provided implementation, the CoT prompt requests step-by-step reasoning and the sampling uses temperature 0.7 with a fixed maximum generation length (max_tokens). For each rationale r_i , we extract a final numeric answer a_i with regex-based parsing.

Second, it forms answer hypotheses by grouping rationales that share the same extracted final answer. For each unique answer a , we obtain a group $G(a)$ of supporting rationales. Let U be the number of unique extracted answers among the K samples.

Third, it verifies each answer hypothesis once using a deterministic verifier prompt (temperature 0.0). For each candidate answer a , the verifier is shown (i) the original problem q , (ii) the proposed answer a , and (iii) up to L representative rationales from $G(a)$, selected as the shortest rationales in that group. The verifier is explicitly instructed to try to DISPROVE the proposed answer using exactly three independent sanity checks: magnitude or estimation, an alternative quick recomputation, and a constraint check (such as bounds, parity, or story consistency). The verifier response is parsed to obtain: (1) confidence $c(a)$, a real number in $[0, 1]$; and (2) checks_passed $p(a)$, an integer in $\{0, 1, 2, 3\}$.

Fourth, it computes a score for each hypothesis and returns the highest-scoring answer. The aggregation rule is multiplicative and exponentiated:

$$\text{score}(a) = \left(\frac{|G(a)|}{K} \right)^\alpha \cdot (c(a))^\beta \cdot \left(\frac{p(a)}{3} \right)^\gamma.$$

In the executed proposed configuration, $\alpha = 0.5$, $\beta = 1.0$, and $\gamma = 1.0$. Compared with SC majority vote (which corresponds to $\alpha = 1$ and omits verification terms), this choice down-weights the impact of raw frequency so that a frequent hypothesis can be penalized if it fails falsification checks or receives low verifier confidence. The `checks_passed` term is normalized by 3 to map it into $[0, 1]$. If parsing fails, the parser defaults `checks_passed` to 0 and confidence to 0.5 and clips both to valid ranges.

The key design choice is verifying answer hypotheses rather than individual chains. Whereas per-chain verification would require K verifier calls, SFw-SC-CoT requires U verifier calls. The method therefore targets settings in which K samples collapse to a small number of distinct answers, so that selective verification can focus compute on discriminating among hypotheses rather than scoring every chain.

Algorithm 1 SFw-SC-CoT (Selective Falsification-Weighted Self-Consistency)

```

1: Input: problem  $q$ , sampler, verifier, sample count  $K$ , reps limit  $L$ , exponents  $\alpha, \beta, \gamma$ 
2: Sample CoT rationales  $\{r_i\}_{i=1}^K$  with stochastic decoding
3: Extract answers  $a_i \leftarrow \text{Extract}(r_i)$  for  $i \in \{1, \dots, K\}$ 
4: Group rationales by answer:  $G(a) \leftarrow \{r_i : a_i = a\}$  for each unique  $a$ 
5: for each unique answer  $a$  do
6:   Select representatives  $R(a) \leftarrow \text{shortest min}(L, |G(a)|)$  rationales from  $G(a)$ 
7:    $(c(a), p(a)) \leftarrow \text{Verify}(q, a, R(a))$  ▷ deterministic, falsification checks
8:    $s(a) \leftarrow \left( \frac{|G(a)|}{K} \right)^\alpha \cdot (c(a))^\beta \cdot \left( \frac{p(a)}{3} \right)^\gamma$ 
9: end for
10: Return  $\arg \max_a s(a)$ 

```

5. Experimental Setup

We evaluate SFw-SC-CoT against a standard self-consistency baseline on GSM8K. This section describes only the experimental conditions that were actually executed and recorded in the provided run configurations and Weights and Biases (WandB) summaries.

5.1. Task and dataset

The task is to answer multi-step math word problems with a single numeric output. We use the GSM8K dataset, main subset, test split. The dataset is loaded via the HuggingFace datasets interface under the identifier `openai/gsm8k` with subset `main`. The run configuration sets `max_samples` to 200, and the logs indicate that exactly $N = 200$ test problems were processed.

5.2. Model and access method

Both baseline and proposed methods use the same model: gpt-4o-mini with base_model gpt-4o-mini-2024-07-18 accessed through the OpenAI chat completions API. The codebase wraps the API in an `LLMModel` class that issues a separate generation call for each prompt.

5.3. Generation and sampling protocol

For each problem, both methods generate $K = 8$ chain-of-thought samples using temperature 0.7 and max_tokens 512. Inference uses batch_size 1. Each completion is

post-processed using the `extract_final_answer` function described in the Background section.

5.4. Methods compared

Baseline SC majority vote. Verification is disabled. After sampling and answer extraction, the baseline selects the answer with the largest group size $|G(a)|$. In the configuration, aggregation parameters are $\alpha = 1.0$, $\beta = 0.0$, and $\gamma = 0.0$, representing frequency-only aggregation.

Proposed SFw-SC-CoT. Verification is enabled. For each unique answer hypothesis a , the method runs a deterministic verifier (temperature 0.0) using a falsification prompt. The verifier receives at most $L = 2$ representative rationales per hypothesis (`max_rationales_per_hypothesis = 2`), chosen as the shortest rationales in that group. The parsed verifier outputs provide confidence and `checks_passed` signals that are combined with frequency using $\alpha = 0.5$, $\beta = 1.0$, and $\gamma = 1.0$.

5.5. Evaluation metric

The primary metric is exact-match accuracy of the final numeric answer. Predictions and labels are normalized as described in Background, and a prediction is counted correct if the normalized strings match.

5.6. Logging and artifacts

Both runs log cumulative accuracy, `num_correct`, and `num_total` to WandB. The evaluation script fetches run summaries and produces four PDF figures: one per-run accuracy trace for each method and two comparison plots across methods.

6. Results

This section reports the logged results from the executed GSM8K runs. No additional datasets, K values, or baselines beyond those described in Experimental Setup were run, and we do not report any unlogged measurements.

6.1. Overall accuracy on GSM8K

On $N = 200$ test problems with $K = 8$, the SC baseline achieves accuracy 0.93, corresponding to 186 correct out of 200. SFw-SC-CoT achieves accuracy 0.94, corresponding to 188 correct out of 200. This is an absolute gain of 0.01 accuracy, or 2 additional problems solved on this evaluation subset.

Expressed in terms of error reduction on this sample, the number of errors decreases from 14 (SC) to 12 (SFw-SC-CoT), which is a 14.3% reduction relative to the baseline error count.

6.2. Statistical strength of the observed difference

With $N = 200$, uncertainty in accuracy estimates is substantial. Using the normal approximation to the binomial standard error, $\sqrt{p(1-p)/N}$, yields approximately 0.018 at $p = 0.93$ and approximately 0.017 at $p = 0.94$, implying wide and overlapping 95% intervals. Because the results are only available as aggregate counts and the paper does not include paired per-item outcomes, we do not perform a paired significance test. The improvement should therefore be interpreted as a small, positive directional result rather than conclusive evidence of a robust advantage.

6.3. Runtime comparison

The WandB summaries report wall-clock runtime. The SC run has runtime 1801 seconds, while the SFw-SC-CoT run has runtime 4495 seconds. Under this implementation

and configuration, the added verification stage increases runtime by roughly $\sim 2.5\times$. The logs do not include the number of unique hypotheses U per problem, the number of verifier calls, or token counts for generation versus verification; therefore, we cannot compute accuracy per call or accuracy per token from these runs.

6.4. Figures

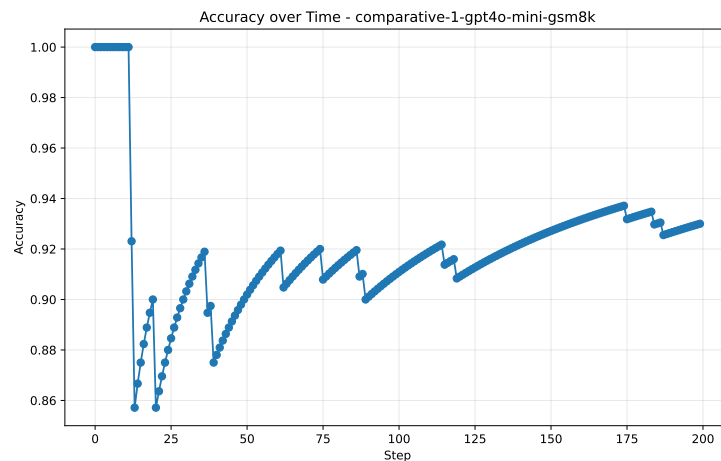


Figure 1. Accuracy over time for SC baseline on GSM8K ($N = 200$); higher values indicate better performance.

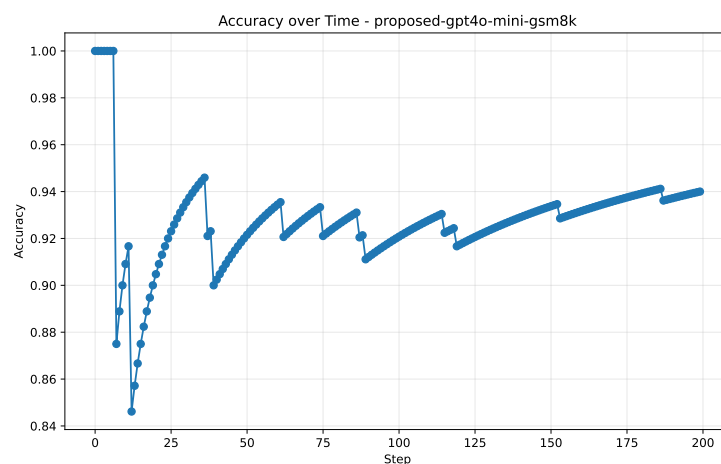


Figure 2. Accuracy over time for SFw-SC-CoT on GSM8K ($N = 200$); higher values indicate better performance.

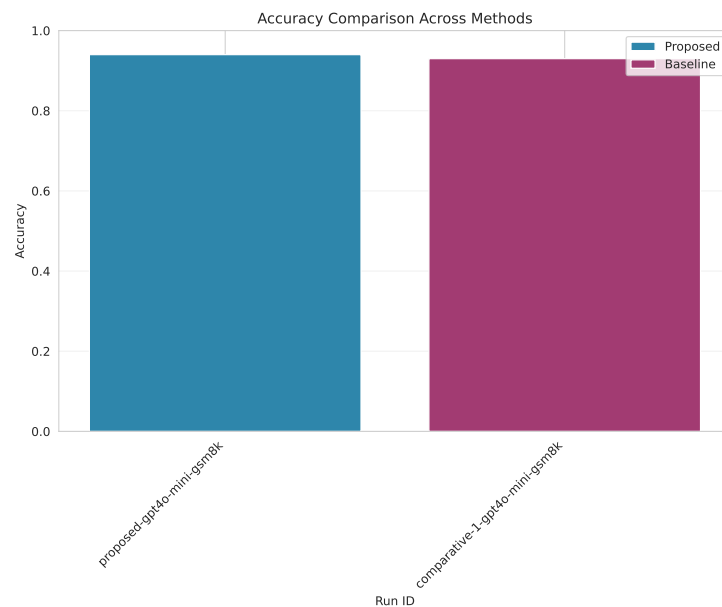


Figure 3. Final accuracy comparison across methods on GSM8K ($N = 200$); higher values indicate better performance.

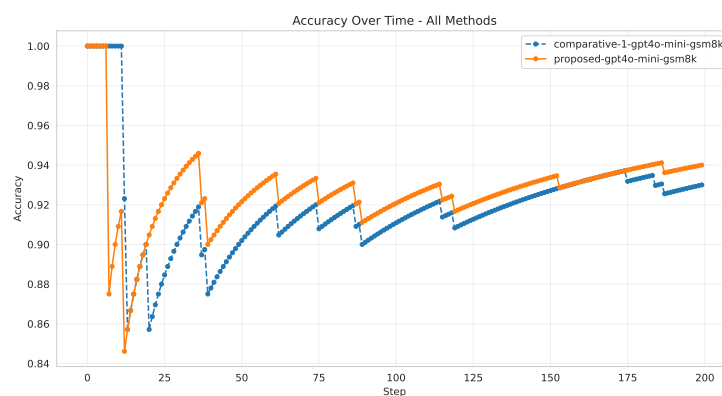


Figure 4. Accuracy over time overlay for SC vs SFw-SC-CoT on GSM8K ($N = 200$); higher values indicate better performance.

The final accuracies visualized in Figure 3 match the WandB summaries (0.93 for SC and 0.94 for SFw-SC-CoT). The per-run traces (Figure 1 and Figure 2) and overlay plot (Figure 4) report accuracy as examples are processed.

6.5. Limitations evidenced by the results

The experiments validate that SFw-SC-CoT can improve accuracy over SC in this specific setting, but they do not yet validate the proposed compute-efficiency motivation. The observed runtime increase, together with missing logs for U and token usage, means we cannot demonstrate a budget-aware tradeoff empirically. Additionally, because verifier outputs (confidence and checks passed) are not logged, we cannot directly evaluate whether falsification signals are well calibrated or whether they discriminate correct versus incorrect answer hypotheses in the intended way.

7. Discussion

8. Conclusions

This paper examined a brittleness of standard self-consistency for chain-of-thought reasoning: majority voting treats sampled chains as exchangeable and uses answer frequency

as a proxy for correctness, which can fail when samples share systematic misconceptions. To address this, we proposed Selective Falsification-Weighted Self-Consistency (SFw-SC-CoT), a prompt-only and inference-only method that clusters sampled chains by extracted final answer and verifies each unique answer hypothesis using a falsification-style checklist consisting of estimation, alternative recomputation, and constraint checking. The verifier produces a continuous confidence score and a discrete checks-passed signal, and these are combined with down-weighted frequency in a multiplicative scoring rule.

On GSM8K test problems ($N = 200$) using gpt-4o-mini with $K = 8$ sampled chains, SFw-SC-CoT improved accuracy from 0.93 (186/200) to 0.94 (188/200). This is a modest but positive directional improvement consistent with the idea that structured falsification can help resolve cases where frequency alone is misleading.

The current evaluation also highlights clear next steps. The logs do not report hypothesis counts, verifier-call counts, or token usage, and the observed runtime increase indicates that engineering and measurement are needed to substantiate the intended budget-aware benefits. Future work should expand the evaluation across datasets and sampling budgets and instrument the pipeline to quantify accuracy as a function of explicit compute cost, while retaining the prompt-only nature of the approach.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used: “Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript”, please turn to the [CRediT taxonomy](#) for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: This research received no external funding.

Data Availability Statement: All resources used in this study are openly available at

Acknowledgments: In this study, we automatically carried out a series of research processes—from hypothesis formulation to paper writing—using generative AI.

Conflicts of Interest: The authors declare no conflicts of interest.

1. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models **2022**.
2. Das, D.; Levy, M.; Pal, P.B.; Prasad, A.M.; Saha, I.; Srivastava, A. Self-Consistency Improves Chain of Thought Reasoning in Language Models **2022**. <https://doi.org/10.1103/PhysRevD.107.055035>.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.